Übungsblatt 5: Ereignisse, Queues und Speicherbesitz

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Eine FIFO durch den Ring verfolgen (8 Punkte)

Ein Ring mit vier Plätzen startet leer; `head = tail = count = 0`. Head bezeichnet den nächsten Get-Platz, Tail den nächsten Put-Platz, beide modulo vier. Ein erfolgreicher Put erhöht Count, Get verringert ihn.

Verfolgen Sie: Put 7, Put 9, Get, Put 12, Put 15, Put 18, Get, Put 21.

1. Geben Sie nach jedem Schritt Head, Tail, Count und die logisch gültige FIFO-Folge an. (5 Punkte)
2. Was liefern die beiden Gets? Wieso bedeutet Head gleich Tail nicht eindeutig leer? (2 Punkte)
3. Ein weiterer Put mit Timeout null trifft auf den Endzustand. Was geschieht, und darf ein fehlgeschlagener Get als neue Nachricht verarbeitet werden? (1 Punkt)

Musterlösung: Zustände `(head,tail,count; Folge)`: (0,1,1; 7), (0,2,2; 7,9), (1,2,1; 9), (1,3,2; 9,12), (1,0,3; 9,12,15), (1,1,4; 9,12,15,18), (2,1,3; 12,15,18), (2,2,4; 12,15,18,21). Gets liefern 7 und 9. Head gleich Tail kommt bei Count null und vier vor. Der weitere Try-Put wird als voll abgewiesen; ein fehlgeschlagener Get liefert keinen neuen gültigen Datensatz.

Aufgabe 2: Ankunft, Bedienung und Überlauf (8 Punkte)

Eine FIFO hat fünf Plätze und startet leer. Bei 2, 4, …, 24 ms wird je eine Nachricht mit Sequenz 1, 2, …, 12 angeboten. Gets bei 8, 16 und 24 ms entnehmen je ein Element. Bei Gleichzeitigkeit gilt **Get vor Put**. API-Zeit null, bei Vollbelegung wird die neue Nachricht verworfen.

1. Bestimmen Sie den ersten Fehl-Put und alle verworfenen Sequenzen. (2 Punkte)
2. Nennen Sie entnommene Werte, erfolgreiche Puts und Restfolge. Prüfen Sie $I = A + D$ und $A = G + Q$: I zählt Angebote, A erfolgreiche Puts, D Drops, G Gets und Q verbliebene Nachrichten. (2 Punkte)
3. Welche Kapazität genügt nur für dieses Intervall? Kann ein endlicher Puffer bei dauerhaft denselben Raten jeden Verlust verhindern? (2 Punkte)
4. Zwei neue Anwendungen haben eine Quelle, die nicht auf die Software wartet: Ein Display benötigt nur den jüngsten gültigen Wert; ein Ereignisarchiv verlangt jeden erfassten Datensatz. Welche Überlaststrategie passt jeweils? Begründen Sie, wann das Ersetzen älterer Werte zulässig ist. (2 Punkte)

Musterlösung: Erster Fehl-Put bei 14 ms, Sequenz 7. Drops: 7, 9, 10, 11. Gets: 1, 2, 3. Acht Puts gelingen; Restfolge 4,5,6,8,12 mit fünf Elementen. $12 = 8 + 4$ und $8 = 3 + 5$. Ein verlustfreier Referenzlauf erreicht maximal neun Plätze, etwa nach Put bei 22 ms und nach Put bei 24 ms. Neun Plätze genügen für dieses endliche Intervall. Dauerhaft 500 Angebote/s gegenüber 125 Entnahmen/s erzeugen 375 zusätzliche Nachrichten/s; ein endlicher Puffer löst die dauerhafte Überlast nicht.

Für das Display dürfen ältere Zustände durch einen jüngeren ersetzt werden, sofern der Übergabevertrag dies ausdrücklich vorsieht und der zusammengehörige Datensatz konsistent gelesen wird. Eine Queue der Länge eins mit gewöhnlichem Try-Put ersetzt ihren alten Inhalt nicht automatisch. Beim Ereignisarchiv verletzt das stille Verwerfen alter oder neuer Datensätze die Anforderung. Die Last muss begrenzt oder die Verarbeitung ausreichend dimensioniert werden; eine trotzdem erkannte Überlast verlangt einen sichtbaren Fehlerpfad. Einen Softwarethread auf Platz warten zu lassen, hält die hier unabhängige Quelle nicht an.

Aufgabe 3: Datensatz oder Pufferadresse? (8 Punkte)

1. Eine Wertqueue kopiert `{seq=6, value=37}`. Der Sender setzt seine lokale Variable danach auf 88. Was erhält der Consumer? Was ändert sich bei einer Queue reiner Pointer? (2 Punkte)
2. Sechs Pointerplätze zu vier Byte stehen drei Bildpuffern zu je 48 KiB gegenüber. Einer wird gelesen, zwei sind eingereiht. Wie viele unabhängige neue Bilder können befüllt werden? Berechnen Sie Pointer-Nutzspeicher und Bildspeicher getrennt. (3 Punkte)
3. Wer besitzt einen Puffer nach fehlgeschlagenem Pointer-Put? Warum genügen `static` oder zwei Get-Consumer nicht für „beide Empfänger bekommen jedes Bild“? (3 Punkte)

Musterlösung: Die Wertqueue enthält die kopierten Bytes mit Wert 37. Ein Pointer kopiert nur eine Adresse; Datenlebensdauer und Überschreibschutz brauchen eigene Besitzregeln. Kein Bildpuffer ist frei. Slots: $6 \cdot 4 = 24$ Byte, Bilder $3 \cdot 48 = 144$ KiB bzw. 147456 Byte; Verwaltung kommt hinzu. Nach fehlgeschlagenem Put bleibt der Producer Besitzer und muss erneut versuchen oder korrekt freigeben. `static` sichert Lebensdauer, keine Exklusivität. Zwei Get-Consumer teilen Nachrichten auf; für beide Pflichtempfänger braucht es eigene Zustell-/Besitzverträge.

Aufgabe 4: Ein Ereignis bis zur Workerreaktion (8 Punkte)

1. Vier NVIC-Prioritätsbits, passende Gruppierung, logische Kernelaufrufgrenze sechs: Bewerten Sie Queue Put mit Timeout null aus IRQ-Priorität fünf und sieben. Welche Zahl bezeichnet die höhere IRQ-Dringlichkeit? Darf ein erlaubter IRQ einen Mutex erwerben? (2 Punkte)
2. Gemeinsame Modelluhr in µs: Ereignis 40, ISR-Eintritt 45, Worker Ready 52, ISR-Ende 58, Workerstart 65, Reaktion 79. Berechnen Sie Eintrittslatenz, ISR-Intervall, Ready-Warten und Gesamtantwort. Prüfen Sie $D = 35\,\mu s$ und vermeiden Sie Doppelzählung. (3 Punkte)
3. Unterscheiden Sie Tasterprellen, Zusammenfassung mehrerer Hinweise in einem Bit und Drop einer bereits erfassten Queue-Nachricht. (1 Punkt)
4. Ein Messregister kann vor dem Workerstart bereits einen neuen Wert enthalten. Der bei ISR-Eintritt vorliegende Wert soll für eine spätere Auswertung erhalten bleiben. Welche Arbeit muss kurz in der ISR erfolgen, welche kann der Worker übernehmen? Genügt ein Weckbit mit späterem Registerlesen? (2 Punkte)

Implementierungshinweis ohne Punkte: Im hier vorgegebenen Vier-Bit-Modell entspricht die logische Grenze sechs dem Registerwert $6 \cdot 2^{8-4} = 96 = 0x60$. Der HAL-Prioritätssetter erhält den logischen Wert sechs.

Musterlösung: Die kleinere logische NVIC-Zahl bezeichnet die höhere IRQ-Dringlichkeit. IRQ fünf ist zu dringlich, sieben darf die ISR-zulässige Put-Funktion mit gültigem Handle und Timeout null verwenden. Mutex Acquire ist auch dort unzulässig. Eintritt 5 µs, ISR 13 µs, Ready-Warten 13 µs, Gesamtantwort 39 µs, Miss um 4 µs. Disjunkt etwa $5 + 13 + 7 + 14 = 39$ µs; Ready-Warten 52–65 überlappt die ISR bis 58. Prellen erzeugt mehrere elektrische Übergänge; ein Bit kann mehrere Set-Aufrufe zusammenfassen; Queue-Drop lehnt eine erfasste Nachricht wegen Vollbelegung ab. Diese Zahlen sind keine automatische Zählung entprellter Tastendrücke.

Die ISR muss den flüchtigen Registerinhalt kurz sichern und beispielsweise als kleine Wertnachricht mit Kennung und bei Bedarf Zeitmarke über eine erlaubte Queue-Funktion ohne Warten übergeben. Auch deren Rückgabestatus und Überlaufpfad gehören zum Übergabevertrag. Umfangreiche Auswertung, Formatierung und verzögerte LED-Ausgabe übernimmt der Worker. Ein Weckbit speichert keine Messbytes; späteres Registerlesen kann bereits einen anderen Wert liefern. Selbst eine korrekte Queueübergabe garantiert nicht, dass jedes physische Ereignis den ISR-Eintritt erreicht hat.

Aufgabe 5: Zwei Queue-Einträge, derselbe Puffer

Eine Queue nimmt zweimal den Pointer auf dasselbe statische Array an. Beim ersten Put enthält es `[3,8]`, vor dem zweiten Put wird es auf `[5,13]` geändert. Der Consumer liest erst danach beide Einträge. Welche Inhalte sieht er, und wie sähe eine korrekte Zwei-Puffer-Übergabe aus?

Musterlösung: Zweimal `[5,13]`: Beide Adressen bezeichnen dieselben inzwischen geänderten Bytes. Für zwei unabhängige Stände werden zwei freie Puffer benötigt, die nach erfolgreichem Put bis zur Verbraucherrückgabe nicht wiederbefüllt werden. Wertkopien hätten beide ursprünglichen Stände erhalten.

Aufgabe 6: Tasterereignisse an einen LED-Worker übergeben

Programmieren, etwa 35–45 Minuten. Die CM4-Basis `lab05_button.c/.h` und `button_student.c` enthält eine FIFO mit drei Plätzen und genau einen Worker. Nur der eingebaute **B1/PC13** erzeugt Ereignisse; **LD1/PB0** zeigt bearbeitete Betätigungen. FreeRTOS/CMSIS-RTOS2 läuft mit 1000 Ticks/s. Der vorhandene EXTI-Callback übergibt seine Pinmaske mit `Lab05_ButtonIRQ(pin)`; dieser geprüfte Einstieg ruft für B1 Ihre Funktion auf.

1. Implementieren Sie `Lab05_QueueButton()` für den IRQ: Datensatz `Lab05_Press` aus fortlaufender Sequenz und `osKernelGetTickCount()` bilden, Try-Put mit Nachrichtenpriorität null und Timeout null. Zählen Sie Annahmen, volle-Queue-Drops und sonstige Fehler getrennt. In der ISR weder warten noch LEDs verzögert bedienen.
2. Implementieren Sie `Lab05_PulseWorker(argument)`: blockierender Get, Flag-/Statusverwechslungen vermeiden. Der erste Datensatz zählt; weitere mit weniger als 40 Tickwerten Abstand zum letzten vom Worker als Betätigung gewerteten Ereignis werden als Prellen ignoriert. Vergleichen Sie die erfassten Ereigniszeitpunkte, nicht die spätere Bearbeitungszeit.
3. Für jeden angenommenen Tastendruck LED einschalten, `osDelay(60U)`, ausschalten und `osDelay(100U)`. Prüfen Sie API-Fehler über `Lab05_ButtonCheck(status)`; Queue-Vollbelegung im IRQ wird gezählt und nicht als normaler Erfolg behandelt.
4. Prüfen Sie einzelne und schnelle Betätigungen. Zählen elektrische Flanken, erreichte Callbacks, erfolgreiche Puts und begonnene LED-Pulse notwendigerweise dasselbe? Interpretieren Sie die Diagnosewerte erst an einem konsistenten Prüfpunkt.

Vorbereitete Basis: Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.

Musterlösung:

Musterlösung zu Aufgabe 6

Die ISR erzeugt nur kleine Wertnachrichten. Entprellung im Worker ist eine festgelegte Unterrichtsregel; sie garantiert bei beliebigem Prellen weder genau einen Puls noch unbegrenzte Ereigniskapazität.

```
#include "lab05_button.h"

void Lab05_QueueButton(void)
{
    const Lab05_Press press = {
        .seq = ++lab05_irq_offered, .tick = osKernelGetTickCount()
    };
    const osStatus_t status = osMessageQueuePut(lab05_press_queue, &press, 0U, 0U);
    if (status == osOK) { ++lab05_irq_accepted; }
    else if (status == osErrorResource) { ++lab05_irq_dropped; }
    else { ++lab05_irq_errors; }
}

void Lab05_PulseWorker(void *argument)
{
    (void)argument;
    int have_last = 0;
    uint32_t last = 0U;
    for (;;) {
        Lab05_Press press;
        Lab05_ButtonCheck(osMessageQueueGet(lab05_press_queue, &press,
                                            NULL, osWaitForever));
        ++lab05_worker_handled;
        if (have_last && (uint32_t)(press.tick - last) < 40U) {
            ++lab05_worker_ignored;
            continue;
        }
        last = press.tick;
        have_last = 1;
        ++lab05_worker_pulses;
        Lab05_ButtonLed(1);
        Lab05_ButtonCheck(osDelay(60U));
        Lab05_ButtonLed(0);
        Lab05_ButtonCheck(osDelay(100U));
    }
}
```

Erfasstes Prellen kann die Queue füllen, bevor der Worker aussortiert. Die Pulszahl zählt begonnene Pulse, nicht jede elektrische Flanke. IRQ-Zähler haben nur den IRQ als Schreiber, Worker-Zähler nur den Worker. Lesen: Quelle sperren, Queue kontrolliert abarbeiten und CM4 anhalten. Im laufenden Mehrfeld-Snapshot wäre `volatile` keine Synchronisation.